2026-2027 9. Sınıf Programlamanın Temelleri

İçindekiler

1. Öğrenme Birimi Algoritmik Düşünme ve Tasarım 1
   1.1. Problem Analizi ve Bileşenleri 1
      1.1.1. Algoritma ve Tasarım Kriterleri 1
      1.1.2. Problemi Bileşenlerine Ayırma 1
      1.1.3. Girdi-İşlem-Çıktı Modeli 1
   1.2. Akış Şeması (Flowchart) 2
      1.2.1. Sözde Kod ve Elle Çalıştırma 2
         1.2.1.1. Örnek 1: İki Sayının Toplamını Bulma (Sayfa 25 Örnek 1) 2
         1.2.1.2. Karşılaştırma Operatörleri ve Kullanım Örnekleri 3
            1.2.1.2.1. Karşılaştırma Operatörleri İle Yapılan İşlem Örnekleri 3
         1.2.1.3. Örnek 1 Hatalı Giriş Kontrolü Algoritması (Sayfa 28 Örnek 4) 4
         1.2.1.4. Örnek 2 Sıcaklık Kontrolü (Sayfa 28 Örnek 3) 5
         1.2.1.5. Örnek 3: Yaş Kontrolü ve Ehliyet Alabilme (Sayfa 25 Örnek 2) 6
         1.2.1.6. Flowgorithm ve Thonny Uygulamaları 7
         1.2.1.7. Değişkenler ve Veri Tipleri 8
            1.2.1.7.1. Temel Veri Tipleri 8
            1.2.1.7.2. Değişken İsimlendirme Kuralları 8
      1.2.2. Akış Şeması (Flowchart) Sembolleri ve Görsel Modelleme 9
         1.2.2.1. Örnek 1: İki Sayının Toplamını Bulma (Sayfa 25 Örnek 1) 9
         1.2.2.2. Örnek 2: İki Sayının Ortalamasını Bulma 9
         1.2.2.3. Örnek Çalışma Soruları 10
         1.2.2.4. Karar Yapıları 10
         1.2.2.5. Örnek 3 Sıcaklık Kontrolü 11
         1.2.2.6. Örnek 4 Hatalı Giriş Kontrolü Algoritması 12
         1.2.2.7. Örnek 5: Yaş Kontrolü ve Ehliyet Alabilme 13
         1.2.2.8. Örnek Çalışma Soruları 14
         1.2.2.9. Döngü(Loop) 15
         1.2.2.10. Örnek 6 Döngüsel Ekran Yazımı (Sayfa 37 Örnek 8) 15
         1.2.2.11. Örnek 7 Ardışık Sayıların Toplamı (Sayfa 38 Uygulama 4) 16
         1.2.2.12. Örnek Çalışma Soruları 18
      1.2.3. Algoritmalarda Metinsel Veriler ve İşlemler 19
         1.2.3.1. Örnek Metin Birleştirme (Sayfa 42 Örnek 9) 19
      1.2.4. Dijital Depo 19

2026-2027 9. Sınıf Programlamanın Temelleri

1. Öğrenme Birimi Algoritmik Düşünme ve Tasarım

1.1. Problem Analizi ve Bileşenleri

ANALİZ
Bir olayı veya durumu incelemek ve ayrıntılı biçimde anlamaya çalışmaktır.

Programlama sürecinde problem analizi, çözülmesi gereken sorunu dikkatlice inceleme aşamasıdır. Bu aşamada sorunun ne olduğu, çözüm için hangi bilgilere ihtiyaç duyulduğu, bu bilgilerle neler yapılacağı ve sonunda hangi sonucun elde edilmek istendiği belirlenir. Örneğin okul kantinindeki sıra sorununda öğrenci sayısı ve alışveriş süreleri incelenir. Daha sonra bekleme süresini azaltacak bir sıra düzeni planlanır.

1.1.1. Algoritma ve Tasarım Kriterleri

Algoritma, belirli bir problemi çözmek veya hedeflenen bir amaca ulaşmak için tasarlanan, açık ve sıralı adımlardır. Algoritmadaki adımlar açık olmalı, sırayla uygulanmalı ve bir sonuca ulaşmalıdır.

Örnek: İki sayının toplamını bulma;

1. Birinci sayıyı al.
2. İkinci sayıyı al.
3. İki sayıyı topla.
4. Sonucu ekrana yazdır.

Örneğin girilen sayılar 8 ve 5 ise, ekranda 13 sonucu gösterilir.

1.1.2. Problemi Bileşenlerine Ayırma

Problemi Bileşenlerine Ayırma, büyük ve karmaşık bir problemi daha küçük ve yönetilebilir parçalara bölme yöntemidir. Her parça ayrı ayrı incelenir ve çözülür. Bu yöntem, hataları bulmayı, sistemi geliştirmeyi ve güncellemeyi kolaylaştırır.

Örnek: Dijital oyun tasarlama : Oyunun karakter hareketleri, sesleri ve puan sistemi ayrı bölümler olarak hazırlanır. Bu bölümler daha sonra birleştirilir. Böylece oyunu geliştirmek ve hataları bulmak kolaylaşır.

1.1.3. Girdi-İşlem-Çıktı Modeli

Bilgisayarda problemler genellikle Girdi–İşlem–Çıktı modeliyle çözülür.

Girdi: Sisteme verilen bilgiler veya verilerdir.

İşlem: Bu bilgiler üzerinde yapılan hesaplama ve kontrollerdir.

Çıktı: İşlem sonunda elde edilen sonuçtur.

Örnek: Bankamatikten para çekme Kullanıcının kartı, şifresi ve çekmek istediği para miktarı girdidir. Bankamatik şifreyi kontrol eder ve yeterli bakiye olup olmadığına bakar; bu bölüm işlemdir. Para, makbuz ve ekranda gösterilen yeni bakiye ise çıktıdır.

1.2. Akış Şeması (Flowchart)

Akış şeması, bir algoritmanın kod yazılmadan önce semboller ve çizimler kullanılarak gösterilmesidir. Çözüm adımlarını daha kolay anlamayı, kontrol etmeyi ve hataları kodlamaya başlamadan fark etmeyi sağlar.

1.2.1. Sözde Kod ve Elle Çalıştırma

Sözde kod, bir algoritmayı herhangi bir programlama dilinin kurallarına bağlı kalmadan, günlük dile yakın ve açık komutlarla yazma yöntemidir. Sözde kodlar, programcının mantık hatalarına odaklanmasını kolaylaştırır.

Dry run(Elle Çalıştırma), bir algoritmayı kodlamadan önce kağıt üzerinde adım adım çalıştırma yöntemidir. Bu sırada değişkenlerin değerleri takip edilir ve sonucun doğru olup olmadığı kontrol edilir. Böylece mantık hataları erken fark edilir.

1.2.1.1. Örnek 1: İki Sayının Toplamını Bulma (Sayfa 25 Örnek 1)

Kullanıcı tarafından girilen iki sayının toplamını ekrana yazdıran sözde kodu ve elle çalıştırma tablosunu hazırlayın. Birinci ve ikinci sayı için kullanıcıdan alınan değerler için sırasıyla **s1** ve **s2** değişkenlerini, toplam için **toplam** değişkenini kullanın.

Sözde Kod

1. Adım: BAŞLA.
2. Adım: Birinci sayıyı (**s1**) kullanıcıdan al.
3. Adım: İkinci sayıyı (**s2**) kullanıcıdan al.
4. Adım: **toplam = s1 + s2** işlemini yap.
5. Adım: toplam **sonucunu ekrana yazdır**.
6. Adım: DUR.

Elle Çalıştırma Tablosu

Örnek Girdiler: s1 = 15, s2 = 25 **Beklenen Çıktı:** toplam = 40

Adım	İşlem	s1	s2	toplam	Açıklama
1	BAŞLA	-	-	-	Algoritma başlar
2	**s1** değerini oku.	**15**	-	-	Kullanıcı 15 sayısını girer
3	**s2** değerini oku.	15	**25**	-	Kullanıcı 25 sayısını girer
4	**toplam = s1 + s2**	15	25	**40**	15 + 25 = 40 işlemi gerçekleşir
5	toplam değerini yazdır	15	25	40	Ekrana 40 sonucu yazılır
6	DUR	15	25	40	Algoritma başarıyla sonlanır

1.2.1.2. Karşılaştırma Operatörleri ve Kullanım Örnekleri

Algoritmalar; günlük hayatta olduğu gibi belirli koşullara göre farklı yollara sapabilir. Yazılımcılar, iki veriyi birbiriyle kıyaslayan ve aralarındaki mantıksal ilişkiyi belirleyen **karşılaştırma operatörlerini** kullanarak **Doğru**(True) veya **Yanlış**(False) mantıksal sonucunu hesaplar.

Aşağıda ki tabloda sık karşılaştırma operatörleri ve kullanımı gösterilmektedir.

Operatör	Anlamı	Örnek Kullanım	Bilgisayar Dilindeki Karşılığı ve Açıklama
==	Eşittir	tahmin == 1234	**tahmin** değeri, **1234** sayısına eşitse sonuç **Doğru** olur.
!=	Eşit Değildir	sifre != “123”	Şifre, “123” değerine eşit değilse sonuç **Doğru** olur.
>	Büyüktür	yas > 18	Yaş değeri, 18’den büyükse sonuç **Doğru** olur.
<	Küçüktür	sicaklik < 0	Sıcaklık, 0’dan küçükse sonuç **Doğru** olur.
>=	Büyük Eşittir	puan >= 50	Puan, 50 veya 50’den daha büyükse sonuç **Doğru** olur.
<=	Küçük Eşittir	hiz <= 90	Hız, 90 veya 90’dan daha küçükse sonuç **Doğru** olur.

1.2.1.2.1. Karşılaştırma Operatörleri İle Yapılan İşlem Örnekleri

EŞİTTİR (==)
- **5 == 5** → Sonuç: true (1)
- **12 == 20** → Sonuç: false (0)
- **“Tuna” == “Tuna”** → Sonuç: true (1)
- **“Ahmet” == “ahmet”** → Sonuç: false (0) (Büyük/küçük harf duyarlılığı)
- **–10 == 10** → Sonuç: false (0)

EŞİT DEĞİLDİR (!=)
- **10 != 5** → Sonuç: true (1)
- **7 != 7** → Sonuç: false (0)
- **“Java” != “Python”** → Sonuç: true (1)
- **“Okul” != “Okul”** → Sonuç: false (0)
- **–4 != 4** → Sonuç: true (1)

BÜYÜKTÜR (>)
- **15 > 8** → Sonuç: true (1)
- **3 > 10** → Sonuç: false (0)
- **5 > 5** → Sonuç: false (0)
- **–2 > –10** → Sonuç: true (1)
- **0.5 > 0.2** → Sonuç: true (1)

KÜÇÜKTÜR (<)
- **4 < 9** → Sonuç: true (1)
- **25 < 12** → Sonuç: false (0)
- **7 < 7** → Sonuç: false (0)
- **–15 < –3** → Sonuç: true (1)
- **1.25 < 1.30** → Sonuç: true (1)

BÜYÜK EŞİTTİR (>=)
- **18 >= 18** → Sonuç: true (1)
- **20 >= 15** → Sonuç: true (1)
- **6 >= 10** → Sonuç: false (0)
- **0 >= 0** → Sonuç: true (1)
- **–5 >= –5** → Sonuç: true (1)

KÜÇÜK EŞİTTİR (<=)
- **5 <= 5** → Sonuç: true (1)
- **3 <= 8** → Sonuç: true (1)
- **14 <= 9** → Sonuç: false (0)
- **–8 <= –2** → Sonuç: true (1)
- **100 <= 99** → Sonuç: false (0)

1.2.1.3. Örnek 1 Hatalı Giriş Kontrolü Algoritması (Sayfa 28 Örnek 4)

Kullanıcının önceden belirlediği bir şifre ile sisteme girişini kontrol eden programının sözde kodunu ve elle çalıştırma tablosunu hazırlayın. Sistem şifresi için **sistemKey**, kullanıcı şifresi için **kullaniciKey** kullanılacak.

Sözde Kod

1. Adım: BAŞLA
2. Adım: **sistemKey = 1234**
3. Adım: kullanıcıdan (**kullaniciKey**) değerini al
4. Adım: **EĞER sistemKey == kullaniciKey**
   EVET ise "Sisteme Girer"
   HAYIR ise "Sisteme Giremez!"
5. Adım: BİTİR

Elle Çalışma Tablosu

Örnek Girdiler: sistemKey=1234, kullaniciKey=9876, **Beklenen Çıktı:** Sisteme Giremez

Ad	İşlem	sistemKey	kullaniciKey	Koşul sistemKey == kullaniciKey	Çıktı	Açıklama
1	BAŞLA	-	-	-	-	Algoritma başlar.
2	sistemKey = 1234	**1234**	-	-	-	Kodun içinde sistem şifresi **sistemKey=1234** oluşturuldu
3	kullaniciKey = 9876 Gir	1234	**9876**	-	-	Kullanıcıdan **kullaniciKey=9876** değeri alındı.
4	sistemKey == kullaniciKey	1234	9876	**HAYIR**	**Giriş yapılamaz!**	Sistemde bulunan şifre ile kullanıcının girdiği şifre karşılaştırılır. Kullanıcı sisteme giremez!
5	DUR	1234	9876	-	-	Algoritma başarıyla sonlanır.

Elle Çalışma Tablosu

Örnek Girdiler: sistemKey=1234, kullaniciKey=1234, **Beklenen Çıktı:** Sisteme Girer

Ad	İşlem	sistemKey	kullaniciKey	Koşul sistemKey == kullaniciKey	Çıktı	Açıklama
1	BAŞLA	-	-	-	-	Algoritma başlar.
2	sistemKey = 1234	**1234**	-	-	-	Kodun içinde sistem şifresi **sistemKey=1234** oluşturuldu
3	kullaniciKey = 1234 Gir	1234	**1234**	-	-	Kullanıcıdan **kullaniciKey=1234** değeri alındı.
4	sistemKey == kullaniciKey	1234	1234	**EVET**	**Giriş yapılır.**	Sistemde bulunan şifre ile kullanıcının girdiği şifre karşılaştırılır. Kullanıcı sisteme girer.
5	DUR	1234	1234	-	-	Algoritma başarıyla sonlanır.

1.2.1.4. Örnek 2 Sıcaklık Kontrolü (Sayfa 28 Örnek 3)

Ortam sıcaklığı (**SICAKLIK**) 20 derecenin altına düştüğünde kombiyi çalıştıran, 20 derecenin üzerine çıktığında ise kombiyi durduran bir termostat programının sözde kodunu ve elle çalıştırma tablosunu hazırlayın.

Sözde Kod

1. Adım: BAŞLA
2. Adım: Sıcaklık değeri oku **SICAKLIK**
3. Adım: **EĞER SICAKLIK < 20**
   EVET ise "Kombi Çalıştır"
   HAYIR ise "Kombi Durdu"
4. Adım: BİTİR

Elle Çalışma Tablosu

Örnek Girdiler: SICAKLIK = 18, **Beklenen Çıktı:** Kombi Çalışır

Adım	İşlem	SICAKLIK	Koşul (SICAKLIK < 20)	Çıktı	Açıklama
1	BAŞLA	-	-	-	Algoritma başlar.
2	SICAKLIK oku	**18**	-	-	Ortamdaki sıcaklık değeri okunur, SICAKLIK değerine atanır
3	SICAKLIK < 20	18	**EVET**	**Kombi Çalışır!**	Koşul Evet
4	DUR	18	-	-	Algoritma başarıyla sonlanır.

Örnek Girdiler: SICAKLIK = 22, **Beklenen Çıktı:** Kombi Durur

Adım	İşlem	SICAKLIK	Koşul (SICAKLIK < 20)	Çıktı	Açıklama
1	BAŞLA	-	-	-	Algoritma başlar.
2	SICAKLIK oku	**20**	-	-	Ortamdaki sıcaklık değeri okunur, SICAKLIK değerine atanır
3	SICAKLIK < 20	20	**HAYIR**	**Kombi Durur!**	Koşul Hayır
4	DUR	18	-	-	Algoritma başarıyla sonlanır.

1.2.1.5. Örnek 3: Yaş Kontrolü ve Ehliyet Alabilme (Sayfa 25 Örnek 2)

Kullanıcıdan o günün yılını (**sistemYil**) ve kullanıcıdan doğum yılını (**dogumYil**) alarak kullanıcının 18 yaşından büyük olması durumunda "**Ehliyet Alır!**" küçük olması durumunda "**Ehliyet Almaz!**" yazısını ekrana yazdıran programın sözde kodunu ve elle çalışma tablosunu hazırlayınız.

Sözde Kod

1. Adım: BAŞLA.
2. Adım: Sistem Tarihi (**sistemYil**) değerini oku.
3. Adım: Kullanıcının Doğum Yılı (**dogumYil**) değerini oku.
4. Adım: **Yaş = sistemYil - dogumYil** işlemini yap.
5. Adım: **Eğer Yaş >= 18**
   EVET ise "**Ehliyet Alır!**" mesajını yazdır.
   HAYIR ise "**Ehliyet Almaz!**" mesajını yazdır.
6. Adım: DUR.

Elle Çalışma Tablosu

Örnek Girdiler: Sistem Tarihi (sistemYil) = 2026, Doğum Yılı (dogumYil) = 2005, **Beklenen Çıktı:** Ehliyet alabilirsiniz.

Adım	İşlem	sistemYil	dogumYil	Yaş	Koşul (Yaş >= 18)	Çıktı	Açıklama
1	BAŞLA	-	-	-	-	-	Algoritma başlar.
2	sistemYil oku	**2026**	-	-	-	-	Sistem tarihi alınır.
3	dogumYil oku	2026	**2005**	-	-	-	Kullanıcı 2005 sayısını girer.
4	Yaş = sistemYil - dogumYil	2026	2005	**21**	-	-	2026 - 2005 = 21
5	Yaş >= 18	2026	2005	21	**Evet**	**Ehliyet Alır!**	Koşul doğru → mesaj yazdırılır.
6	DUR	2026	2005	21	-	-	Algoritma başarıyla sonlanır.

Örnek Girdiler: Sistem Tarihi (sistemYil) = 2026, Doğum Yılı (dogumYil) = 2010, **Beklenen Çıktı:** Ehliyet almak için henüz küçüksünüz.

Adım	İşlem	sistemYil	dogumYil	Yaş	Koşul (Yaş >= 18)	Çıktı	Açıklama
1	BAŞLA	-	-	-	-	-	Algoritma başlar.
2	sistemYil oku	**2026**	-	-	-	-	Sistem tarihi alınır.
3	dogumYil oku	2026	**2010**	-	-	-	Kullanıcı 2010 sayısını girer.
4	Yaş = sistemYil - dogumYil	2026	2010	**16**	-	-	2026 - 2010 = 16
5	Yaş >= 18	2026	2010	16	**Hayır**	**Ehliyet Almaz!**	Koşul yanlış → mesaj yazdırılır.
6	DUR	2026	2010	16	-	-	Algoritma başarıyla sonlanır.

1.2.1.6. Flowgorithm ve Thonny Uygulamaları

Derslerimizde kullanacağımız yazılımların kurulum bağlantılarına ve destekleyici eğitim videolarına yan taraftaki karekodları taratarak veya altlarındaki bağlantılara tıklayarak doğrudan erişebilirsiniz.

Flowgorithm Programı

Algoritmik düşünme becerimizi geliştirmek ve tasarladığımız mantıksal adımları görselleştirmek için Flowgorithm yazılımını kullanacağız. Bu program sayesinde yazdığımız akış şemalarını adım adım çalıştırabilir, değişkenlerin değer değişimlerini anlık olarak izleyebilir ve algoritmalarımızı kod yazmadan önce test edebiliriz.

Flowgorithm Programı

Bağlantıya tıklayın
flowgorithm.org

Flowgorithm Youtube Eğitimi

Derste işlediğimiz konuları pekiştirmek ve kendi hızınızda pratik yapabilmeniz için Atilla Çetin tarafından hazırlanan Flowgorithm ile Algoritma ve Akış Diyagramları video serisinden yararlanabilirsiniz. Temel kavramlardan uygulamalı örneklere kadar uzanan bu oynatma listesine sağdaki karekodu okutarak veya bağlantıya tıklayarak ulaşabilirsiniz.

Youtube Eğitimi

Bağlantıya tıklayın
kisa.link/bHGEg

Thonny IDE

Akış şemalarından kodlamaya geçerken, Python öğrenimi için özel olarak tasarlanmış Thonny IDE'yi kullanacağız. Sade arayüzü ve adım adım kod takibi sunan hata ayıklama (debug) özellikleriyle başlangıç seviyesine son derece uygundur. Dahili Python desteği sayesinde harici bir kurulum gerektirmez; yalnızca Thonny'yi kurarak kodlarınızı hemen çalıştırmaya başlayabilirsiniz.

Thonny

Bağlantıya tıklayın
thonny.org

Programlamanın Temelleri Ders Notları

Derslerimizde işlediğimiz konuları düzenli takip edebilmeniz ve tekrar yapabilmeniz için hazırladığım bu doküman; MEB'in 14 Eylül 2026 tarihli **'Programlamanın Temelleri 9'** müfredatını temel almaktadır. Konu anlatımları, akış şemaları ve örnek kod uygulamalarıyla ders içi kazanımları pekiştirmeyi amaçlayan kapsamlı bir çalışma kaynağıdır

Ders Notları

Bağlantıya tıklayın
kisa.link/hpYCg

1.2.1.7. Değişkenler ve Veri Tipleri

DEĞİŞKEN NEDİR?

Programın çalışması sırasında kullanıcıdan alınan, hesaplanan veya geçici olarak hafızada (RAM) tutulması gereken verileri saklayan isimlendirilmiş bellek alanlarıdır.

Flowgorithm üzerinde bir değişkeni kullanmadan önce mutlaka **Tanımlama (Declare)** adımıyla bilgisayara tanıtmak gerekir. Aynı türdeki birden fazla değişken aralarına virgül konularak tek seferde tanımlanabilir (Örn: `sistemYil, dogumYil, yas`).

1.2.1.7.1. Temel Veri Tipleri

Flowgorithm içerisinde en sık kullanılan 4 temel veri tipi bulunur:

Veri Tipi	Türü	Örnek Değerler	Açıklama
Tamsayı (Integer)	Tam Sayı	`15`, `0`, `-8`, `2026`	Küsüratsız, negatif veya pozitif tam sayı değerlerini tutar. Yaş, adet, sayaç gibi veriler için kullanılır.
Reel (Real / Float)	Ondalıklı Sayı	`3.14`, `18.5`, `-0.75`	Kesirli veya ondalıklı sayıları tutar. Ortalama, boy, sıcaklık veya fiyat gibi hassas hesaplamalarda tercih edilir.
String	Metinsel Veri	`"Ahmet"`, `"1234"`, `"A"`	Çift tırnak içinde yazılan her türlü yazı, harf, sembol ve rakamı saklar. **Matematiksel işlemlere girmez.**
Boolean	Mantıksal	`true(1)`, `false(0)`	Yalnızca iki değerden birini alabilir: **Doğru (true(1))** ya da **Yanlış (false(0))**. Karar ve döngü kontrollerinde kullanılır.

1.2.1.7.2. Değişken İsimlendirme Kuralları

Değişken tanımlarken programların hata vermemesi ve kodun düzenli olması için belirli kurallara uymak şarttır.

Türkçe Karakter Kullanımı

Kodlama dillerinin ve yazılım araçlarının küresel standardı gereği, değişken isimlendirirken
Türkçe'ye özgü harfleri kullanmaktan kesinlikle kaçınınız!.
Aşağıdaki tabloda büyük ve küçük Türkçe karakterlerin yerine İngilizce alfabe karşılıkları tercih edilmelidir:

Türkçe Karakter	Doğru Kullanım	İngilizce Karakte
ç / Ç	→	**c / C**
ğ / Ğ	→	**g / G**
ı / İ	→	**i / I**
ö / Ö	→	**o / O**
ş / Ş	→	**s / S**
ü / Ü	→	**u / U**

Camel Case (Deve Notasyonu) Kuralı

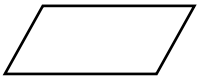
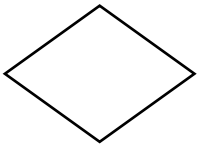
Değişken adları tanımlanırken kelimeler arasında boşluk bırakılamaz. Birden fazla kelimeden oluşan değişkenleri okunaklı kılmak için dünya çapında standart olan **camelCase** yöntemi kullanılır.
Kural: İlk kelimenin tüm harfleri küçük başlar. Sonraki her kelimenin ilk harfi büyük yazılır (kelimelerin ortasındaki hörgüç görünümü gibi).

Yanlış Biçim	camelCase	Açıklama
`sistem yil`	`sistemYil`	Boşluk yerine birleşik yazılır
`dogumyil`	`dogumYil`	İkinci kelimenin baş harfi büyük
`ogrencinotortalamasi`	`ogrenciNotOrtalamasi`	Okunabilirlik artırılır

1.2.2. Akış Şeması (Flowchart) Sembolleri ve Görsel Modelleme

Akış şeması(flowchat), bir algoritmanın adımlarını semboller ve oklarla gösteren görsel bir modeldir. Programın çalışma mantığını kolayca anlamayı ve kodlama öncesinde olası hataları fark etmeyi sağlar.

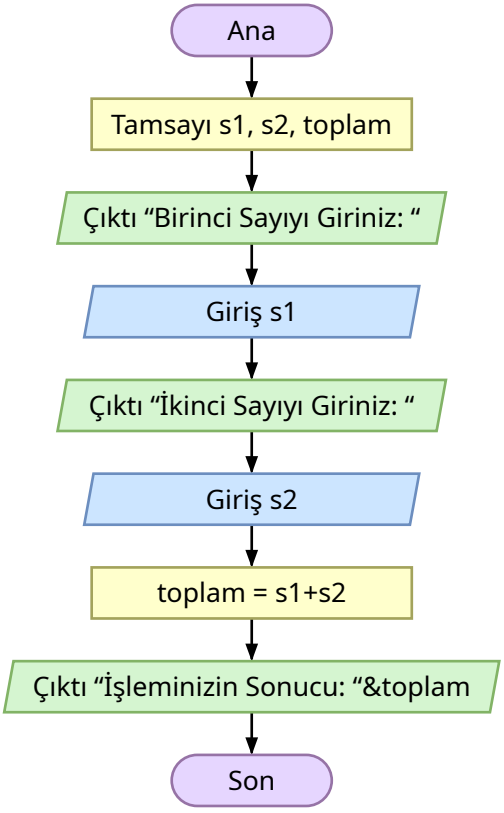
Tablo 1.2: Temel Akış Şeması Sembolleri, İşlevleri ve Günlük Hayat Karşılıkları.

Sembol	Adı	İşlevi	Günlük Hayat Karşılığı
	Oval (Terminal)	Algoritmanın başlangıcını ve bitişini temsil eder.	Bir işe başlama veya sonlandırma anıdır.
	Paralelkenar (Giriş / Çıkış)	Kullanıcıdan veri alınacağını veya ekrana bilgi yazdırılacağını belirtir.	Soru sormak veya cevap vermek içindir.
	Dikdörtgen (İşlem)	Matematiksel hesaplamalar ve değişken atamaları için kullanılır.	Bir eylemi gerçekleştirmek içindir.
	Eşkenar Dörtgen (Karar)	Bir koşulun kontrol edildiği noktadır. Sonuca göre akış Evet veya Hayır yönünde ilerler.	Yol ayrımında karar vermek içindir.
	Oklar (Akış Yönü)	Adımlar arasındaki ilerleme yönünü gösterir.	İzlenecek yolu gösteren oklardır.

1.2.2.1. Örnek 1: İki Sayının Toplamını Bulma (Sayfa 25 Örnek 1)

Kullanıcı tarafından girilen iki sayının toplamını bulan örneğin akış şemasını (flowchart) çiziniz. Birinci ve ikinci sayı ve toplam için sırasıyla **s1**, **s2**, **toplam** değişkenleri kullanılacak.

Akış Şeması(Flowchart)

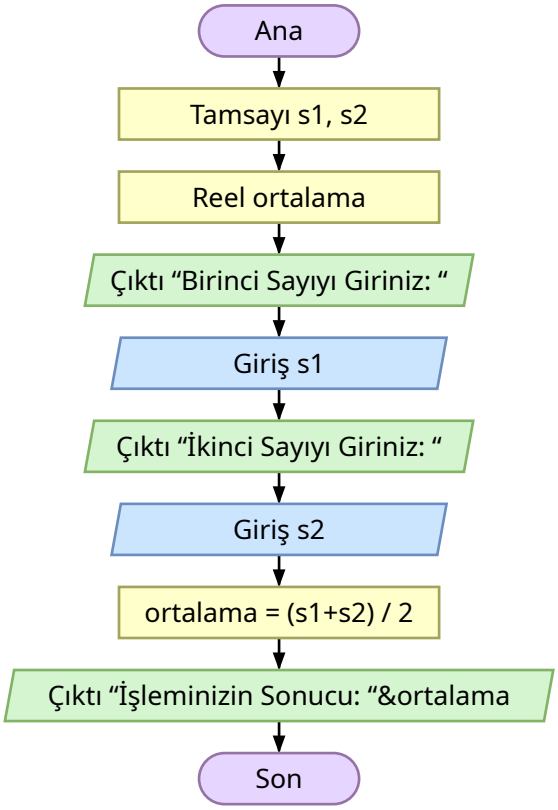


1.2.2.2. Örnek 2: İki Sayının Ortalamasını Bulma

Kullanıcı tarafından girilen iki sayının ortalamasını bulan örneğin akış şemasını(flowchat) çiziniz. Birinci ve ikinci sayı ve ortalama için sırasıyla **s1**, **s2**, **ortalama** değişkenleri kullanılacak.

Ortalama hesabı: `ortalama = (s1 + s2) / 2`

Akış Şeması(Flowchart)

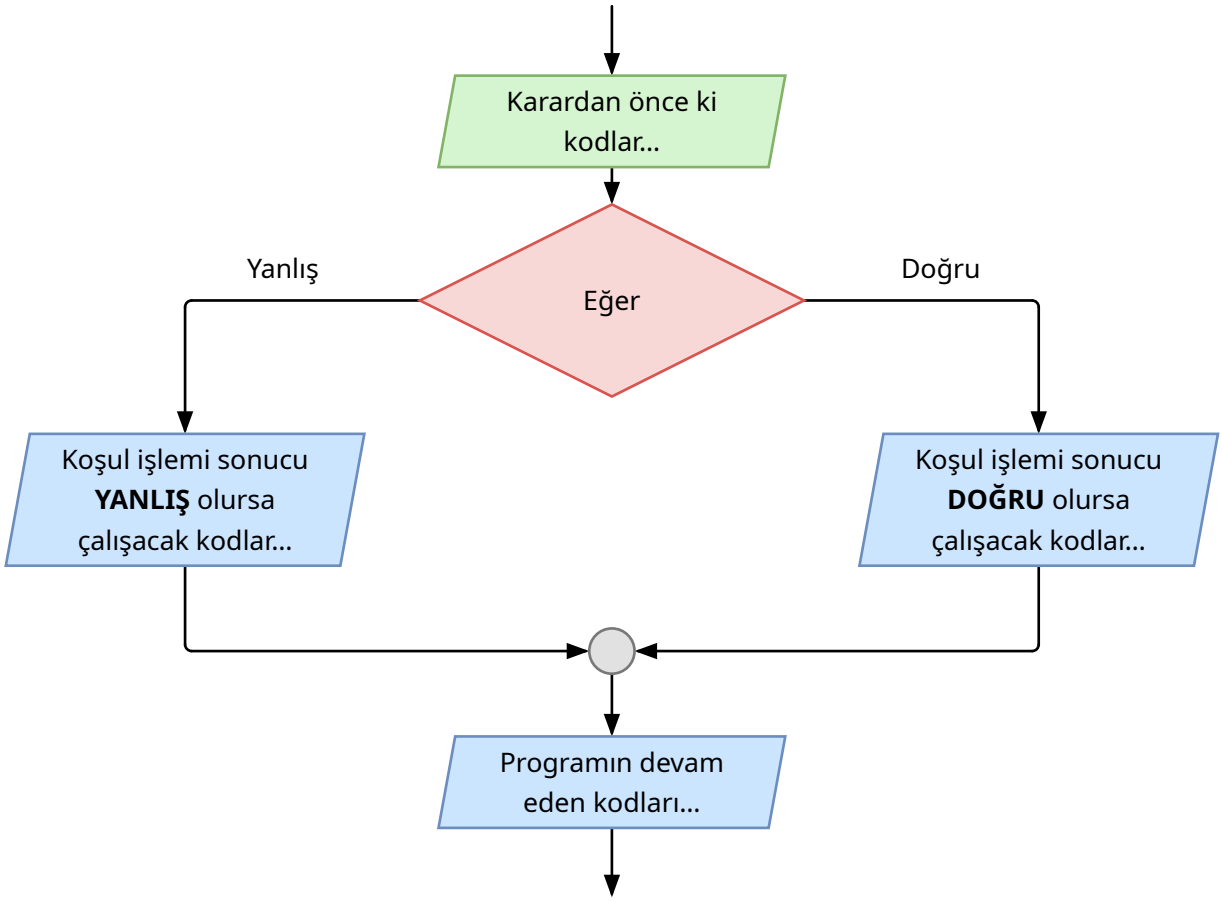


1.2.2.3. Örnek Çalışma Soruları

1. Yaş Hesaplama: Kullanıcıdan doğum yılını ve mevcut yılı alıp yaşını hesaplayan ve ekrana yazdıran akış şemasını çiziniz.
2. Kare Alan ve Çevre: Bir kenar uzunluğu kullanıcıdan girilen karenin alanını ve çevresini hesaplayıp ekrana yazdıran akış şemasını çiziniz.
3. KDV Hesaplama: Fiyatı girilen bir ürünün %20 KDV tutarını ve KDV dahil toplam satış fiyatını hesaplayıp ekrana yazdıran akış şemasını çiziniz.
4. Dakikayı Saniyeye Çevirme: Kullanıcıdan dakika cinsinden bir süre girmesini isteyen, bu sürenin kaç saniye olduğunu hesaplayıp yazdıran akış şemasını çiziniz.
5. Diktörtgenler Prizması Hacmi: Kullanıcıdan en, boy ve yükseklik değerlerini alarak prizmanın hacmini (Hacim = en **boy** yükseklik) hesaplayıp ekrana yazdıran akış şemasını çiziniz.
6. Ağırlıklı Not Ortalaması: Bir öğrencinin 1. vize (%40) ve 2. vize (%60) notlarını alıp dönem sonu başarı puanını hesaplayan ve yazdıran akış şemasını çiziniz.
7. Birim Dönüşümü (Sıcaklık): Kullanıcıdan Celsius cinsinden sıcaklık değerini alıp Fahrenheit değerine çeviren (F = C *1.8 + 32) ve sonucu gösteren akış şemasını çiziniz.
8. Yol, Hız ve Zaman: Bir aracın gittiği mesafeyi (km) ve bu mesafeyi aldığı süreyi (saat) kullanıcıdan alıp ortalama hızını hesaplayan ve ekrana yazdıran akış şemasını çiziniz.
9. Taksi Ücreti Hesaplama: Taksimetre açılış ücreti 30 TL, kilometre başına ücret 20 TL olan bir şehirde; gidilen kilometre bilgisini kullanıcıdan alıp ödenecek toplam tutarı hesaplayıp yazdıran akış şemasını çiziniz.

1.2.2.4. Karar Yapıları

Karar yapıları, programın bir koşulu kontrol ederek hangi adımı izleyeceğini belirler. Akış şemasında eşkenar dörtgen ile gösterilir. Koşul doğruysa bir yola, yanlışsa başka bir yola devam edilir. Altta ki şemada görüldüğü gibi eğer işlemi ile yapılan kontrolün sonunda kod doğru ya da yanlış ihtimale doğru hareket edecektir.



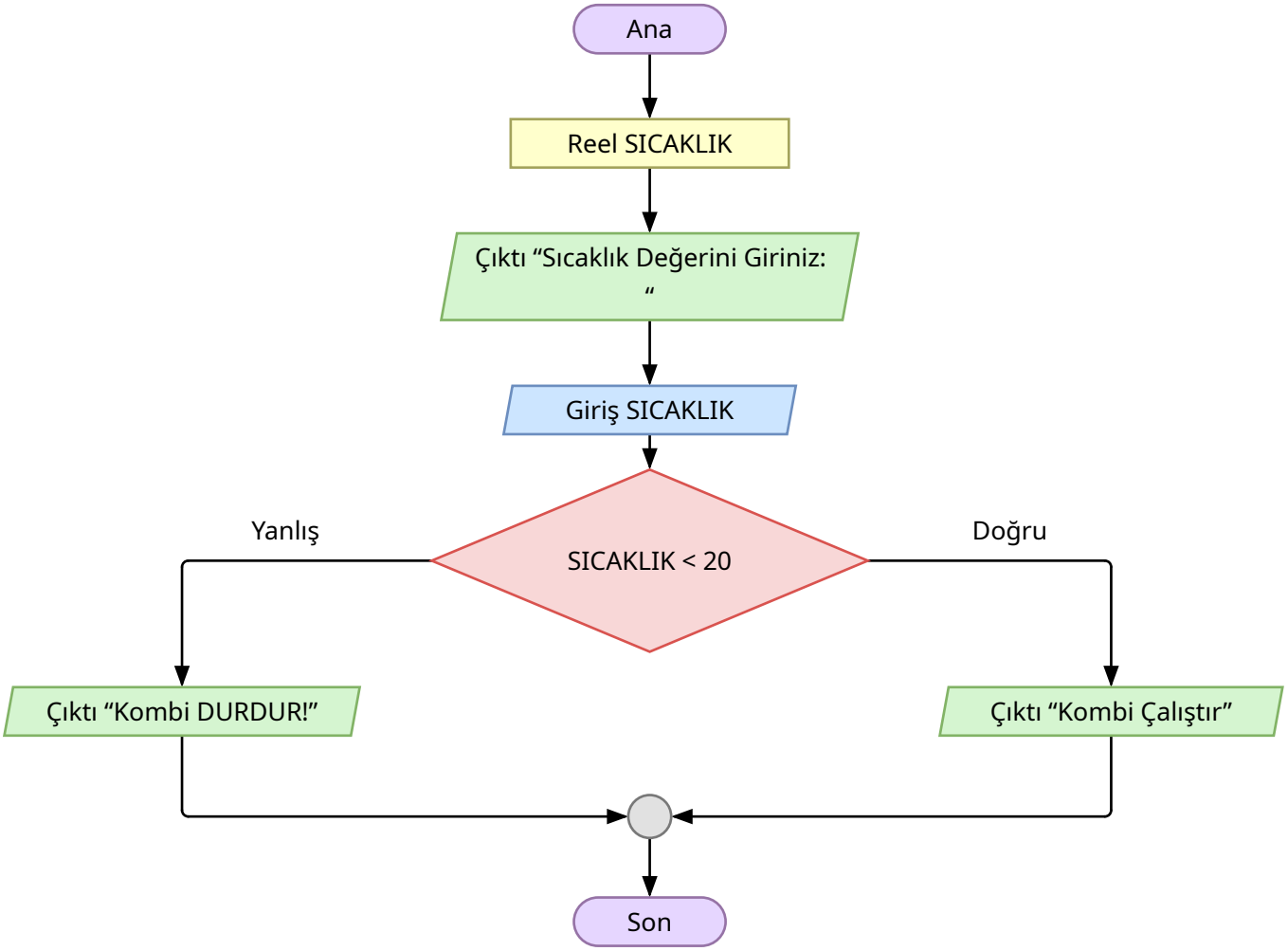
1.2.2.5. Örnek 3 Sıcaklık Kontrolü

Ortam sıcaklığı (**SICAKLIK**) 20 derecenin altına düştüğünde kombiyi çalıştıran, 20 derecenin üzerine çıktığında ise kombiyi durduran bir termostat programının akış şemasını hazırlayın.

Sözde Kod

1. Adım: BAŞLA
2. Adım: Sıcaklık değeri oku **SICAKLIK**
3. Adım: **EĞER SICAKLIK < 20**
   EVET ise “Kombi Çalıştır”
   HAYIR ise “Kombi Durdu”
4. Adım: BİTİR

Akış Şeması(Flowchart)



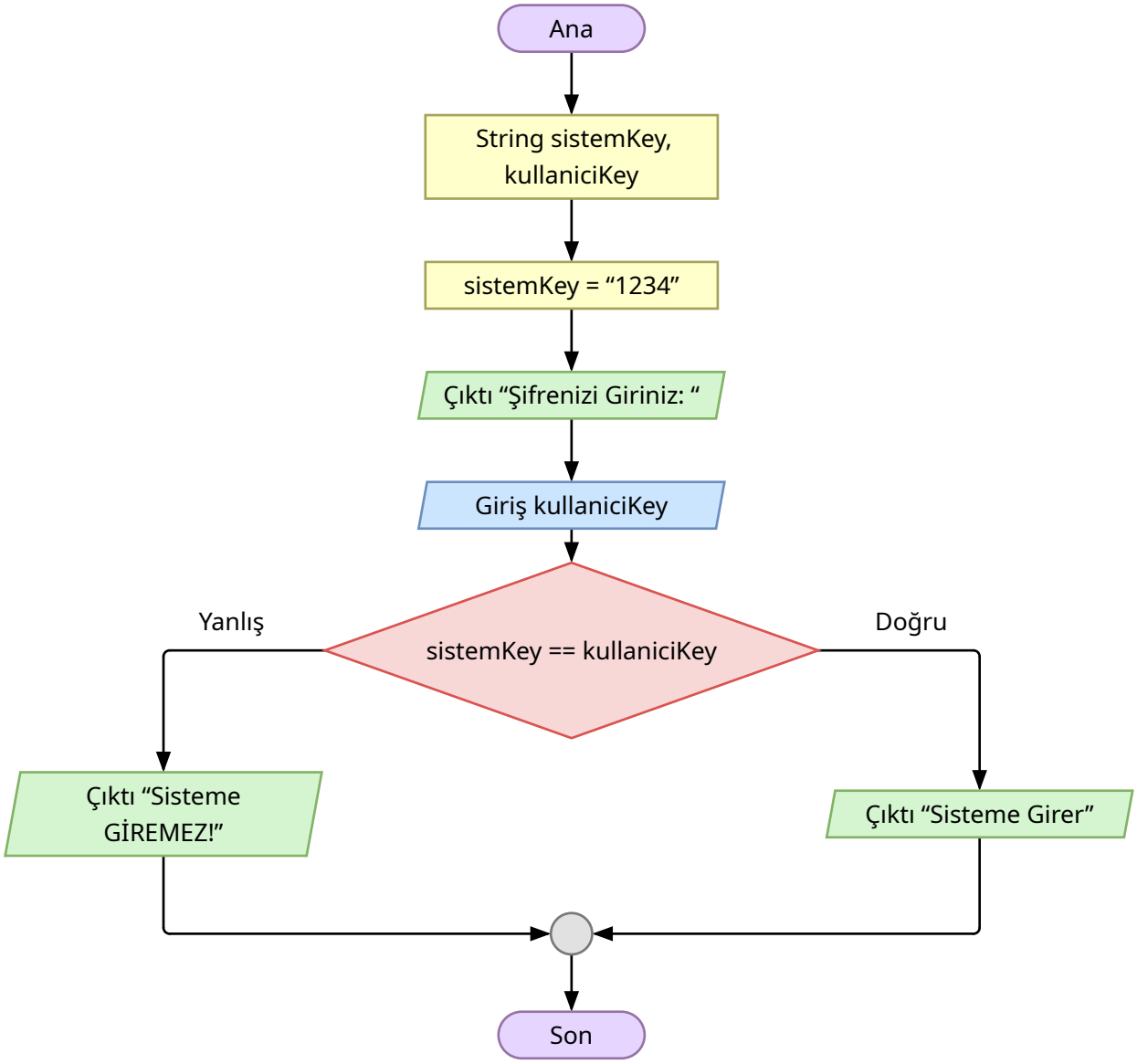
1.2.2.6. Örnek 4 Hatalı Giriş Kontrolü Algoritması

Kullanıcının önceden belirlediği bir şifre ile sisteme girişini kontrol eden programının akış şemasını hazırlayın. Sistem şifresi için **sistemKey**, kullanıcı şifresi için **kullaniciKey** kullanılacak.

Sözde Kod

1. Adım: BAŞLA
2. Adım: **sistemKey = 1234**
3. Adım: kullanıcıdan (**kullaniciKey**) değerini al
4. Adım: **EĞER sistemKey == kullaniciKey**
   EVET ise “Sisteme Girer”
   HAYIR ise “Sisteme Giremez!”
5. Adım: BİTİR

Akış Şeması(Flowchart)



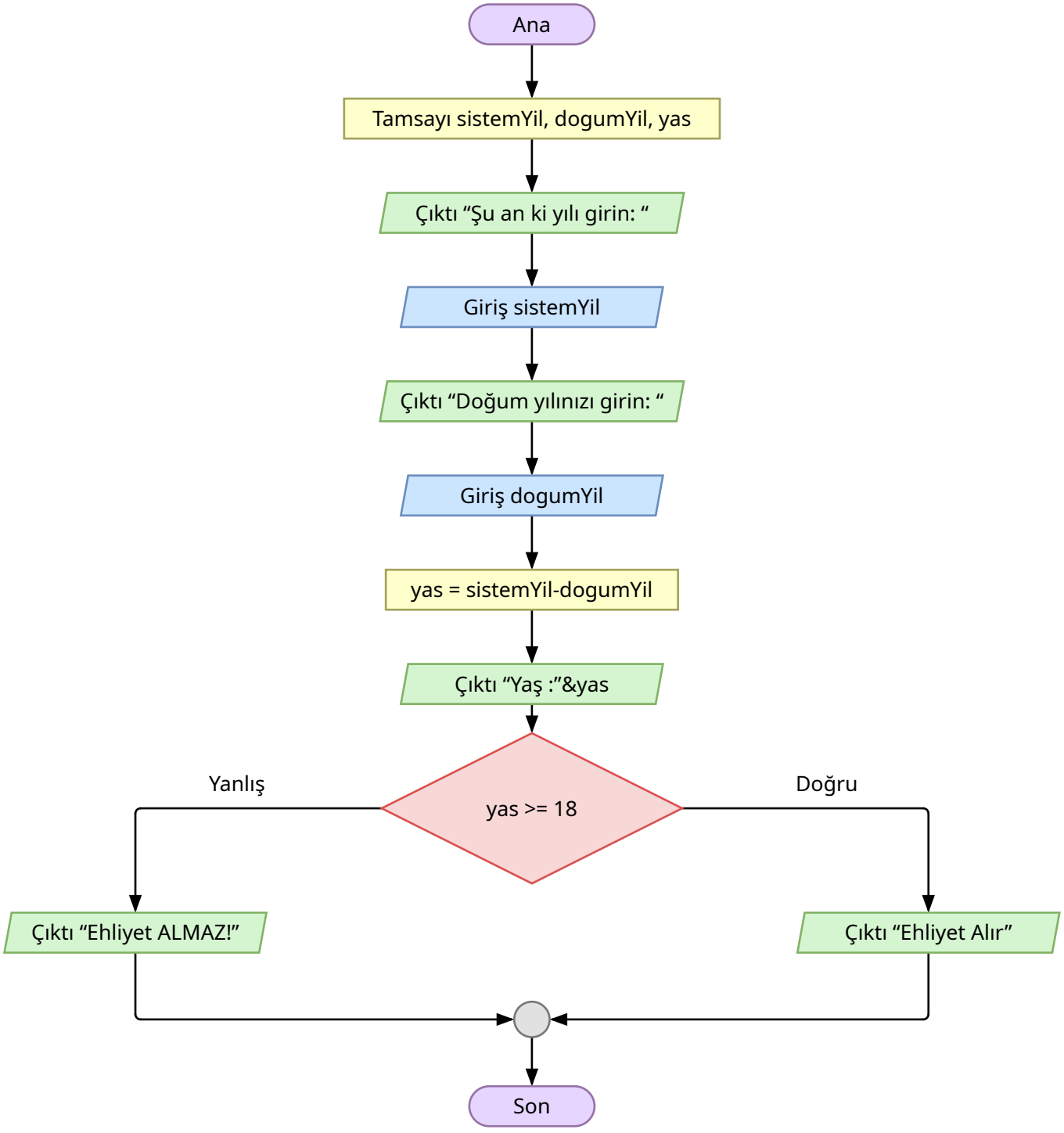
1.2.2.7. Örnek 5: Yaş Kontrolü ve Ehliyet Alabilme

Kullanıcıdan o günün yılını (**sistemYil**) ve kullanıcıdan doğum yılını (**dogumYil**) alarak kullanıcının 18 yaşından büyük olması durumunda "**Ehliyet Alır!**" küçük olması durumunda "**Ehliyet Almaz!**" yazısını ekrana yazdıran programın sözde kodunu ve elle çalışma tablosunu hazırlayınız.

Sözde Kod

2. Adım: Sistem Tarihi (**sistemYil**) değerini oku.
3. Adım: Kullanıcının Doğum Yılı (**dogumYil**) değerini oku.
4. Adım: **yas = sistemYil - dogumYil** işlemini yap.
5. Adım: **Eğer yas >= 18**
   EVET ise "**Ehliyet Alır!**" mesajını yazdır.
   HAYIR ise "**Ehliyet Almaz!**" mesajını yazdır.

Akış Şeması(Flowchart)



1.2.2.8. Örnek Çalışma Soruları

1. Akıllı Bahçe Sulama Sistemi (Sensör ve Eşik Değeri Mantığı): Bir seradaki toprak nem oranı sensöründen gelen değer NEM değişkeninde tutulmaktadır.
   - Nem oranı 30'un altına düştüğünde sulama vanasını açan ("Sulama Başlatıldı"),
   - 30 veya üzerinde olduğunda ise sulama vanasını kapatan ("Sulama Durduruldu") programın akış şemasını çiziniz.
2. Bakiye ve Ödeme Kontrolü (İki Değişkeni Karşılaştırma): Bir online alışveriş sisteminde kullanıcının mevcut hesap bakiyesi BAKIYE, satın almak istediği sepet tutarı ise SEPET değişkenidir.
   - Eğer BAKIYE değeri SEPET tutarına eşit veya büyükse ekrana "Ödeme Başarılı" yazdıran ve kalan yeni bakiyeyi ekranda gösteren,
   - Yetersiz ise ekrana "Yetersiz Bakiye" uyarısı veren programın akış şemasını çiziniz.
3. Vize-Final ile Dersten Geçme / Kalma (İşlem + Karar + Çalışma Tablosu): Bir öğrencinin vize notu VIZE, final notu FINAL olarak kullanıcıdan alınacaktır.
   - Ortalama formülü: ORT = (VIZE **0.4) + (FINAL** 0.6)
   - Eğer ORT değeri 50 veya üzerinde ise ekrana "Geçti", 50'nin altında ise "Kaldı" yazdıran programın sözde kodunu (pseudo-code) ve en az iki farklı öğrenci notu için elle çalışma (izleme) tablosunu hazırlayınız.
4. Hız Sınırı ve Radar Kontrolü (Hız ve Ceza Durumu): Şehir içi yolda hız sınırını kontrol eden radar sistemi, araç hızını HIZ değişkeni olarak okumaktadır. Yasal hız sınırı 70 km/s olarak belirlenmiştir.
   - Sürücünün hızı 70'in üzerindeyse ekrana "Hız Sınırı Aşıldı - Ceza Uygulandı",
   - 70 veya altındaysa ekrana "İyi Yolculuklar" mesajını veren programın akış şemasını çiziniz.
5. Basit Kargo Ücreti Hesaplama (Eşik Değeri ve Şartlı Matematik): Bir e-ticaret sitesinde sipariş tutarı TUTAR değişkeninde tutulmaktadır.
   - Sipariş tutarı 500 TL veya üzerinde ise kargo ücretsizdir (Ödenecek tutar = TUTAR).
   - Sipariş tutarı 500 TL'nin altında ise 50 TL kargo ücreti eklenmektedir (Ödenecek tutar = TUTAR + 50). Kullanıcıya son ödenecek toplam tutarı ekrana yazdıran programın akış şemasını ve sözde kodunu hazırlayınız.
6. Pozitif / Negatif Sayı Tespiti (Temel Sayısal Karar): Kullanıcıdan bir tamsayı (SAYI) girmesini isteyen, girilen sayının 0'dan büyük olması durumunda ekrana "Pozitif Sayı", 0'dan küçük olması durumunda "Negatif Sayı" yazdıran programın akış şemasını hazırlayın.

1.2.2.9. Döngü(Loop)

Bazı işlemlerin belirli bir koşul sağlanana kadar tekrar edilmesi gerekir. Bu tür işlemler için döngüler kullanılır.

Bir sayaç kontrollü döngü üç temel bölümden oluşur:

- **Başlangıç değeri:** Sayacın başlayacağı değerdir.
- **Döngü koşulu:** Döngünün devam edip etmeyeceğini belirler.
- **Sayaç:** Her tur sonunda değeri artırır veya azaltır.

Koşul doğru olduğu sürece döngü devam eder. Koşul yanlış olduğunda döngü sona erer. Sayaç değerinin değiştirilmesi, döngünün sonsuza kadar sürmesini önler.

1.2.2.10. Örnek 6 Döngüsel Ekran Yazımı (Sayfa 37 Örnek 8)

Döngü kullanarak ekrana üç kez “Merhaba Geliştirici!” mesajını yazdırınız.

Sözde Kod

1. Adım: **sayac** tam sayı değişkenini tanımla
2. Adım: sayac = 1 ile sayac değişkenine ilk değeri olan biri ata.
3. Adım: Eğer **sayac <= 3** sayac küçük eşit
   EVET ise

   “Merhaba Geliştirici!” yazdız
   sayac = sayac + 1 => sayac değerini bir arttır

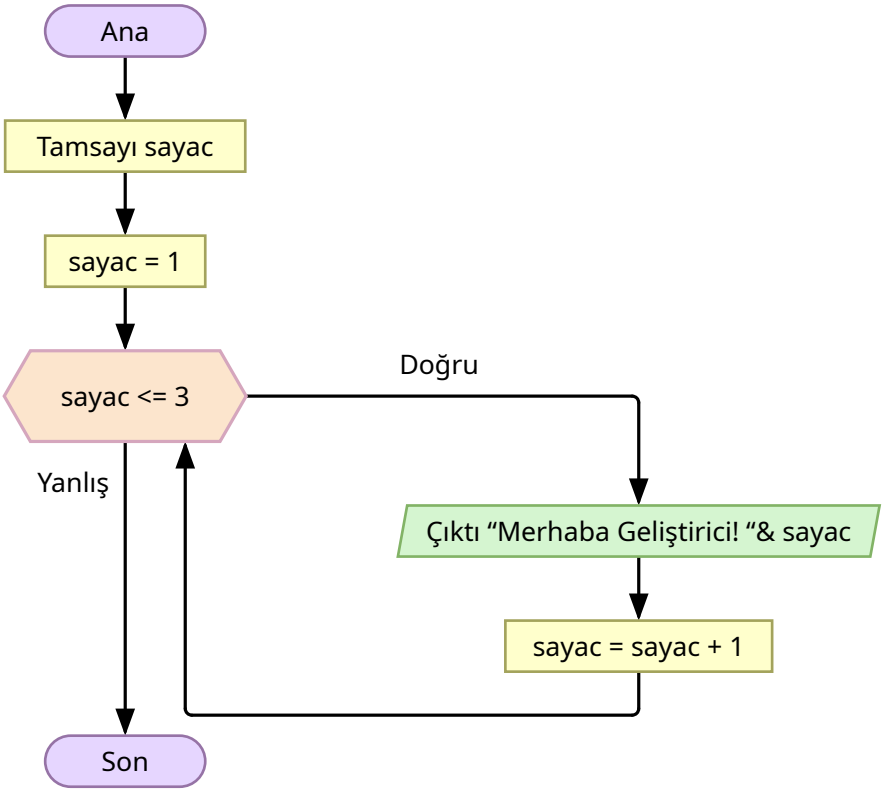
   HAYIR ise

   Döngüden çık

Elle Çalışma Tablosu
Örnek Girdiler: sayac = 3 , **Beklenen Çıktı:** ekrana üç kere “Merhaba Geliştirici” yazdırılacak

Adım	İşlem	sayac	Döngü Koşul (sayac <= 3)	Çıktı	Açıklama
1	Tam sayı **sayac**	-	-	-	sayac değişkeni tanımlanır
2	sayac = 1	1	-	-	sayac değişkenine 1 değeri atanır
3	sayac <= 3	1	1 <= 3 **EVET**	Merhaba Geliştirici	**1<= 3** sonucu **EVET** ekrana yazdırılır sayac 1 arttırılır
4	sayac <= 3	2	2<= 3 **EVET**	Merhaba Geliştirici	**2<= 3** sonucu **EVET** ekrana yazdırılır sayac 1 arttırılır
5	sayac <= 3	3	3<= 3 **EVET**	Merhaba Geliştirici	**3<= 3** sonucu **EVET** ekrana yazdırılır sayac 1 arttırılır
6	sayac <= 3	4	4<= 3 **HAYIR**	-	**4<= 3** sonucu **HAYIR** **Döngü Sonlanır**

Akış Şeması (Flowchart)



Ekran Çıktısı

Merhaba Geliştirici! 1

Merhaba Geliştirici! 2

Merhaba Geliştirici! 3

1.2.2.11. Örnek 7 Ardışık Sayıların Toplamı (Sayfa 38 Uygulama 4)

Birden başlayarak kullanıcının belirlediği değere kadar belirlenen dahil tüm sayıların toplamını alan kodun akış şemasını hazırlayın. **Örnek Hesap : `n = 3 için toplam = 1 + 2 + 3 = 6`**

Sözde Kod

1. Adım: **sayac, toplam ve n** tam sayı değişkenini tanımla
2. Adım: **sayac = 1** ile sayac değişkenine ilk değeri olan biri ata.
3. Adım: **toplam = 0** başlangıç değerini ata
4. Adım: kullanıcıdan **n** değişkenin değerini al
5. Adım: Eğer **sayac <= n** sayac küçük eşit n
   EVET ise

   toplam = toplam + sayac => toplama işlemini yap
   sayac = sayac + 1 => sayac değerini bir arttır

   HAYIR ise

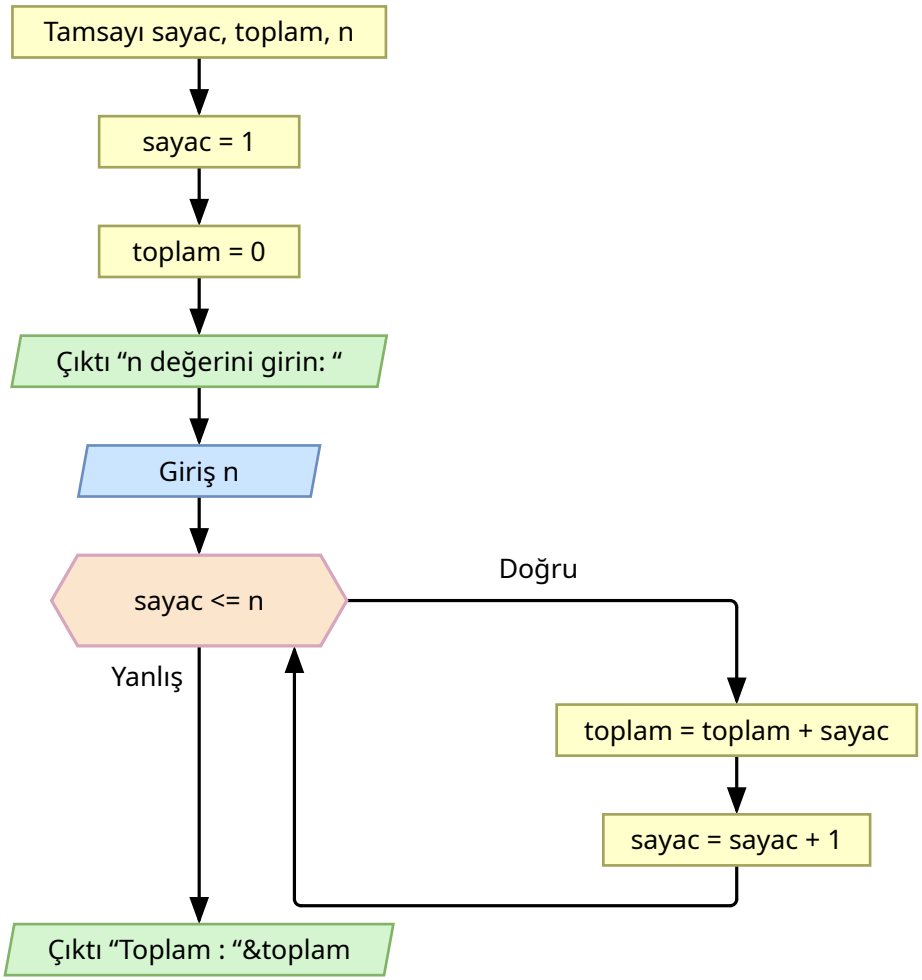
   Döngüden çık
6. Adım: **“Toplam = “&toplam** toplam değerini yazdır.

Elle Çalışma Tablosu

Örnek Girdiler: n = 3 için, **Beklenen Çıktı:** toplam = 6

Ad	İşlem	sayac	toplam	n	Döngü Koşul (sayac <= n)	Çıktı	Açıklama
1	Tam sayı sayac, toplam, n	-	-	-	-	-	Kullanılacak değişkenler tanımlanır
2	sayac = 1, toplam = 0	1	0	-	-	-	sayac ve toplam değişkenleri ilk değerlerini alır
3	n = 3	1	0	3	1 <= 3 **evet**	-	koşul doğru sayac = 2, toplam = 0 + 1 = 1
4	n = 3	2	1	3	2 <= 3 **evet**	-	koşul doğru sayac = 3, toplam =1 + 2 = 3
5	n = 3	3	3	3	2 <= 3 **evet**	-	koşul doğru sayac = 4, toplam = 3 + 3 = 6
6	n = 3	4	6	3	4 <= 3 **HAYIR**	“Toplam :” 6	koşul yanlış sayac = 4, toplam = 6

Akış Şeması (Flowchart)



1.2.2.12. Örnek Çalışma Soruları

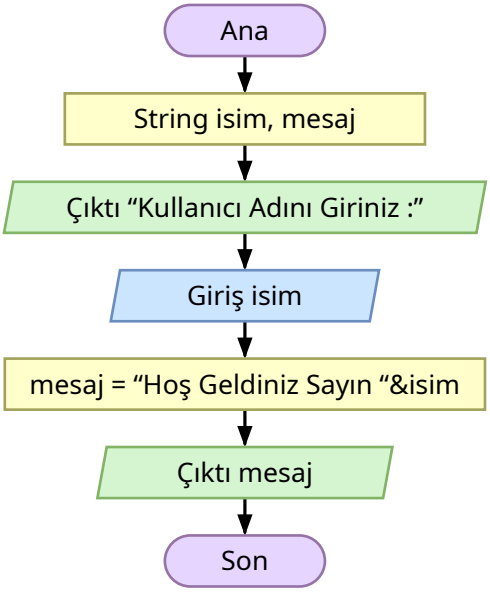
1. 1 ile 100 arasında belirlenen gizli bir sayının (örneğin 33) kullanıcının tahminleriyle bulunmaya çalışıldığı bir tahmin oyununun akış şemasını çiziniz. Döngü kontrolü için boolean tipinde dongu değişkenini kullanınız. Girilen tahmin gizli sayıdan farklı olduğu sürece kullanıcıyı "Sayı Küçük!" veya "Sayı Büyük!" şeklinde yönlendiriniz. Her tahmini sayarak doğru sonuca ulaşıldığında toplam deneme sayısını ekrana yazdırınız.
2. Geri Sayım ve Fırlatma (Geriye Doğru Sayıcı Mantığı): Bir roket fırlatma simülasyonu için 10'dan başlayıp 1'e kadar birer birer geriye doğru sayan ve her sayıyı ekrana yazdıran, döngü bittiğinde ise ekrana "Fırlatma Başarılı!" mesajı veren programın akış şemasını çiziniz.
   İpucu: Döngü sayacını her adımda 1 azaltınız (SAYAC = SAYAC - 1).
3. Çift Sayıların Çarpımı (Faktöriyel / Kümülatif Çarpım Mantığı): Kullanıcıdan pozitif bir çift sayı (N) alarak, 2'den başlayıp N değerine kadar olan (N dahil) tüm çift sayıların çarpımını hesaplayan programın akış şemasını ve sözde kodunu hazırlayınız.
   Örnek Hesap: N = 6 için Carpim = 2 * 4 * 6 = 48
   Öğrenci Notu: Çarpım için başlangıç etkisiz elemanını (CARPIM = 1) ve adım artışını (SAYAC = SAYAC + 2) doğru belirleyiniz.
4. Sınıf Not Ortalaması Hesaplama (Döngü İçinde Girdi Alma ve Ortalama): Kullanıcıdan önce sınıfta kaç öğrenci olduğu bilgisini (ADET) alan, ardından döngü yardımıyla bu adet kadar öğrenci notunu (NOT) teker teker kullanıcıdan isteyip toplayan, döngü sonunda ise sınıfın genel not ortalamasını hesaplayıp ekrana yazdıran programın akış şemasını çiziniz.
   Elle Çalışma: ADET = 3 ve girilen notlar 70, 80, 90 olacak şekilde elle çalışma (izleme) tablosunu oluşturunuz.
5. PIN Kodu ile Telefon Kilidi Açma (3 Hak Sınırlı): Telefonun kayıtlı PIN kodu DOGRU_PIN = 1234 olarak belirlenmiştir. Kullanıcıya en fazla 3 deneme hakkı tanınacaktır.
   Kullanıcı doğru PIN'i girerse ekrana "Kilit Açıldı" yazıp döngü derhal sonlanmalıdır.
   Kullanıcı 3 hakkında da hatalı giriş yaparsa ekrana "Telefon Kilitlendi!" uyarısı verilmelidir.
   Bu senaryoyu gerçekleştiren programın akış şemasını çiziniz.
6. Sıfır Girilene Kadar Pozitif Sayıları Sayma: Kullanıcı klavyeden 0 (sıfır) girene kadar sürekli sayı girişi kabul eden bir program tasarlayınız.
   Girilen sayılar arasından yalnızca pozitif olanların kaç adet olduğunu saymalıdır (POZITIF_ADET).
   Kullanıcı 0 girdiğinde döngüden çıkılmalı ve toplamda kaç adet pozitif sayı girildiği ekrana yazdırılmalıdır.
   Programın sözde kodunu ve 5, –3, 12, –1, 0 test girdileri için elle çalışma tablosunu hazırlamak işinizi kolaylaştırır.

1.2.3. Algoritmalarda Metinsel Veriler ve İşlemler

Bilgisayarlar, sayılar dışında isim ve şifre gibi metinsel verileri de işler. Çift tırnak içinde yazılan metinlere **`string`** (**karakter dizisi**) denir. Metinsel veriler doğrudan matematiksel işlemlerde kullanılamaz.

1.2.3.1. Örnek Metin Birleştirme (Sayfa 42 Örnek 9)

Kullanıcıdan aldığı isim bilgisiyle ekrana “Hoş Geldiniz Sayın :” + isim şeklinde yazı yazdıran kodun akış şemasını çiziniz?



1.2.4. Dijital Depo

Yazılım geliştirme sürecinde kaynak kodların güvenle saklandığı, sürümlerinin takip edildiği alanlara dijital depo (repository) denir. Günümüzde GitHub ve GitLab gibi platformlar; projelerin sürümler halinde yönetilmesini, dağıtılmasını ve ekiplerce ortak geliştirilmesini sağlar. Her deponun vitrini sayılan README.md dosyası ise projenin amacını ve kullanımını özetleyen teknik imzasıdır.

Sizlerin yapacağı yazılım projeleri de ilerleyen derslerimizde GitHub üzerinde yayımlanarak paylaşıma açılacaktır.

Ahmet Tuna POTUR GitHub Hesabı

Geliştirdiğim açık kaynaklı yazılımları, eğitim materyallerini ve projelerin sürüm geçmişlerini incelemek için kişisel GitHub profilimi ziyaret edebilirsiniz.

A.Tuna POTUR GitHub

Bağlantıya tıklayın
github.com/tunapotur

Programlamanın Temelleri 9 GitHub Projesi

Ders notlarımızın kaynak kodlarını barındıran örnek dijital depoyu buradan inceleyebilirsiniz. Projenin amacını öğrenmek için vitrin belgesi olan README.md dosyasını ve geliştirme sürecindeki sürüm adımlarını gözlemleyebilirsiniz.

GitHub Prog. Temelleri

Bağlantıya tıklayın
kisa.link/WeLYU